TP04: Containerization and Big Data

Environment Deployment Using Docker

1 Introduction

The objective of this practical work is to understand Docker containerization and its role in deploying data processing applications.

In this project, a Python application was containerized to analyze a large air quality dataset. Several Docker concepts were used, including images, containers, volumes, networks, port mapping, Docker Hub, and Docker Compose.

The project demonstrates how Docker can provide a portable, isolated, and reproducible environment for data processing tasks.

2 Docker Installation and Verification

Docker was installed and verified successfully on the Linux environment.

The following commands were used to verify the installation:

```
docker --version
docker info
docker compose version
```

A Docker Hub account was also used to publish and retrieve the project image.

3 Docker Image Creation and Distribution

A custom Docker image was created using a Dockerfile based on the official Python 3.11 slim image.

The image was tagged as:

`dhia8448/tp04-air-quality:latest`

The image was uploaded to Docker Hub and successfully pulled into the local Docker environment.

The main commands were:

```
docker build -t dhia8448/tp04-air-quality:latest .

docker push dhia8448/tp04-air-quality:latest

docker pull dhia8448/tp04-air-quality:latest
```

Docker Hub repository:

https://hub.docker.com/r/dhia8448/tp04-air-quality

4 Dockerfile Explanation

The Dockerfile contains the instructions required to build the application image.

The Dockerfile used in this project is:

```
FROM python:3.11-slim

WORKDIR /app

COPY app.py .

CMD ["python", "-u", "app.py"]
```

4.1 FROM

Specifies the base image used to build the new image. In this project, `python:3.11-slim` provides a lightweight Python environment.

4.2 WORKDIR

Defines the working directory inside the container. All subsequent relative paths are evaluated from this directory.

4.3 COPY

Copies application files from the host build context into the Docker image.

Here, `app.py` is copied into the `/app` directory.

4.4 RUN

Executes commands during the image build process. It is commonly used to install system packages or Python dependencies.

For example:

```
RUN pip install pandas
```

The current application uses only Python standard libraries, so a RUN instruction is not required.

4.5 CMD

Defines the default command executed when a container starts.

In this project, it launches the Python analysis application:

```
CMD ["python", "-u", "app.py"]
```

5 Container Creation and Networking

Three containers were created from the downloaded Docker image.

A custom bridge network named `tp04-network` was created to connect the containers.

The containers were configured with the following port mappings:

Table 1: Container Port Mapping

Container	Host Port	Container Port
tp04-container-1	8081	80
tp04-container-2	8082	80
tp04-container-3	8083	80

Port mapping allows services running inside containers to be accessed through ports on the host machine.

The web application was tested successfully using:

- http://localhost:8081
- http://localhost:8082
- http://localhost:8083

All three addresses displayed the project web page successfully.

6 Docker Compose Deployment

A `compose.yaml` file was created to define the data analysis service.

The configuration was:

```
services:
  data-analysis:
    build: .
    image: dhia8448/tp04-air-quality:latest
    container_name: tp04-analysis
    volumes:
```

```
      - /home/dhia-okba/Desktop/air_quality_5gb.csv:/app/data/
        air_quality_5gb.csv:ro
    restart: "no"
```

The configuration contains the following elements:

- **build: .**: Builds the image using the Dockerfile in the current directory.
- **image**: Assigns a name and tag to the built image.
- **container_name**: Specifies the container name.
- **volumes**: Mounts the dataset from the host into the container.
- **:ro**: Makes the mounted dataset read-only.
- **restart: no**: Prevents automatic container restart.

The application was built and launched using:

```
docker compose up --build
```

The deployment completed successfully. The analysis container exited with code 0, indicating successful execution.

7 Dataset Analysis

The application processed a large air quality dataset containing 66,937,826 records.

The dataset has 12 columns, including:

- Timestamp
- Sensor ID
- District
- Temperature
- Humidity
- PM2.5 and PM10
- CO2 and NO2
- Wind speed
- Traffic level
- Air quality index

The Python application used the `csv` module to read the dataset and the `Counter` class to count records by district.

It calculated the average temperature and average air quality index.

7.1 Overall Results

Table 2: Dataset Analysis Results

Metric	Result
Total data rows	66,937,826
Average temperature	22.00 °C
Average air quality index	18.22

7.2 Records by District

Table 3: Number of Records per District

District	Number of Records
Industrial	13,387,565
Centre	11,900,058
North	11,528,184
South	10,784,428
West	10,412,549
East	8,925,042

The analysis completed successfully.

8 Report Questions

8.1 Question 1: What is Docker, and why is it useful in data science projects?

Docker is an open-source platform used to package applications and their dependencies into containers.

It is useful in data science projects because:

- It provides a consistent execution environment.
- It reduces dependency and compatibility problems.
- It makes applications portable between different machines.
- It simplifies deployment and testing.
- It allows applications to run in isolated environments.
- It makes sharing and reproducing experiments easier.

For example, the Python analysis application in this project can run inside a container without requiring Python to be installed directly on every host machine.

8.2 Question 2: What is the difference between a Docker image and a Docker container?

A Docker image is a read-only template that contains the application code, runtime, libraries, and required configuration.

A Docker container is a running or stopped instance created from a Docker image.

The main differences are:

Table 4: Docker Image vs Docker Container

Docker Image	Docker Container
Template used to create containers.	Instance created from an image.
Contains application files and dependencies.	Runs the application in an isolated environment.
Can be used to create multiple containers.	Represents one application instance.
Image layers are read-only.	A writable container layer is added.

In this project, `dhia8448/tp04-air-quality:latest` is the image, while `tp04-container-1`, `tp04-container-2`, and `tp04-container-3` are containers created from it.

8.3 Question 3: What is the purpose of FROM, WORKDIR, COPY, RUN, and CMD in a Dockerfile?

- **FROM**: Selects the base image.
- **WORKDIR**: Sets the working directory inside the image.
- **COPY**: Copies files from the build context into the image.
- **RUN**: Executes commands during image building.
- **CMD**: Defines the default command executed when a container starts.

Together, these instructions define how the application environment is built and executed.

8.4 Question 4: What problem does Docker Compose solve?

Docker Compose simplifies the management of multi-container applications.

Instead of creating and configuring each container manually, Compose allows developers to define services, networks, volumes, and other settings in a single YAML file.

It makes application deployment easier, repeatable, and more organized.

8.5 Question 5: What is the difference between docker run and docker compose up?

The `docker run` command creates and starts an individual container using command-line options.

The `docker compose up` command reads a Compose configuration file and creates or starts all the services defined in that file.

Table 5: docker run vs docker compose up

docker run	**docker compose up**
Primarily used to start individual containers.	Manages services defined in a Compose project.
Configuration is provided through command-line options.	Configuration is stored in a YAML file.
Useful for simple or one-off containers.	Useful for multi-service applications.
Repeated commands may be required.	Configuration can be reused.

8.6 Question 6: How can we extend a Docker Compose project to include a database?

A database can be added as another service in the `compose.yaml` file.

For example, the following configuration adds a MySQL database:

```
services:
  data-analysis:
    build: .
    depends_on:
      - database

  database:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: example
      MYSQL_DATABASE: air_quality
    volumes:
      - mysql_data:/var/lib/mysql

volumes:
  mysql_data:
```

The database service runs MySQL in a separate container.

The application service can communicate with it using the service name `database` as the hostname.

The named volume `mysql_data` stores database files persistently, even if the database container is recreated.

This is an illustrative extension and was not deployed in this practical work. In a real deployment, database credentials should be stored securely rather than written directly in the Compose file.

9 Conclusion

This practical work demonstrated the main concepts of Docker containerization.

A Python application was packaged into a Docker image, published to Docker Hub, and executed inside multiple containers.

A custom bridge network and port mapping were used to expose web services. Docker Compose was also used to automate the deployment and execution of the data analysis application.

The application successfully processed 66,937,826 air quality records and calculated the required statistics.

This project demonstrates how Docker can provide a portable, isolated, and reproducible environment for data processing applications.